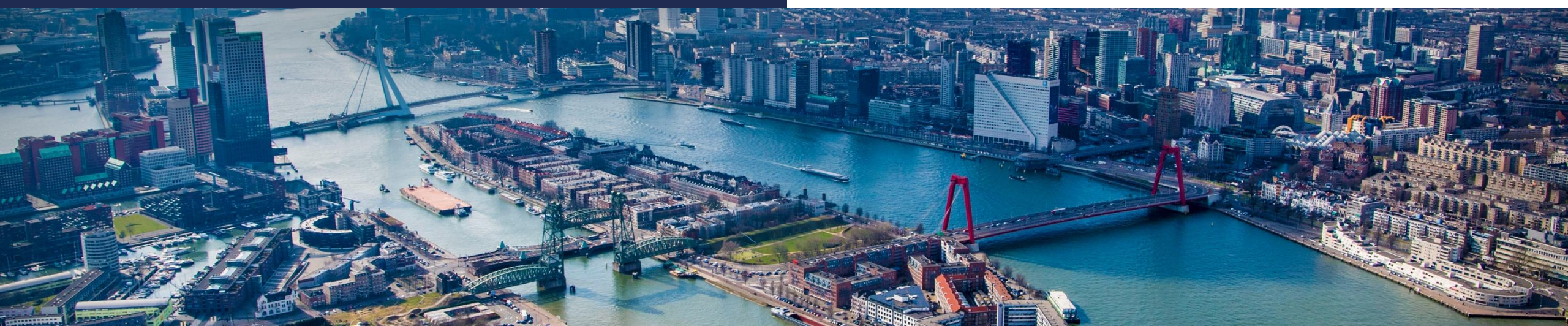


MVP Introductory Session

FinTech: Business Models and Application

Shiwei Ye email: ye@rsm.nl Office: T08-56



Agenda

1

**Timeline &
Logistics**

2

**Assignment &
Requirements**

3

Grading

4

Coding with AI

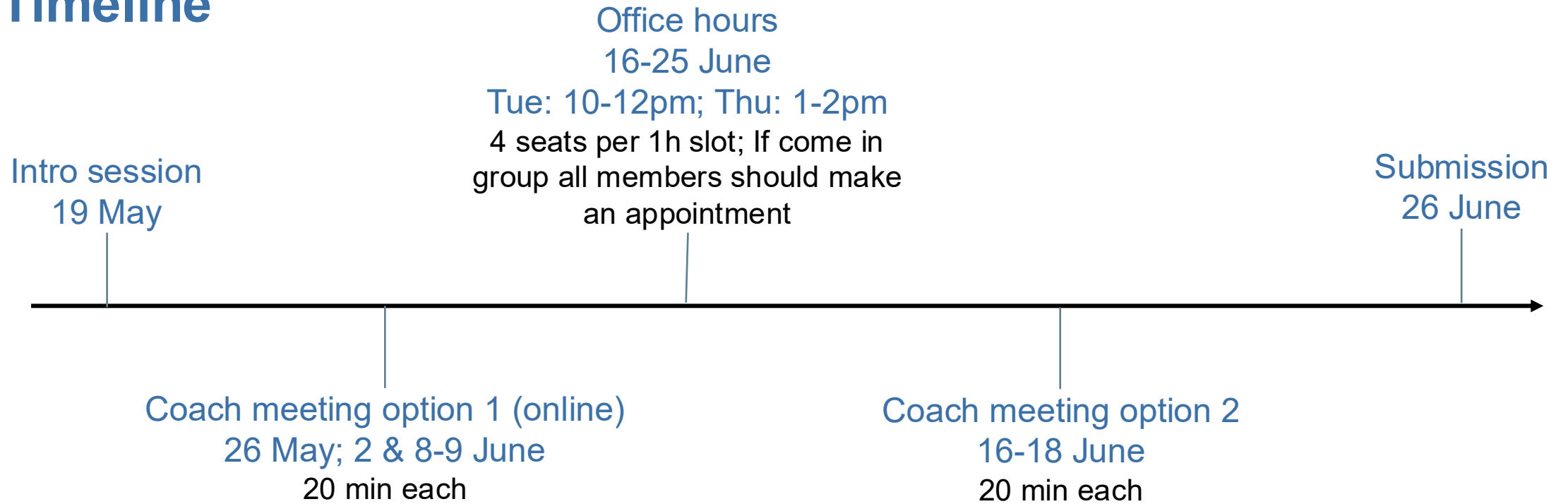
5

Illustration

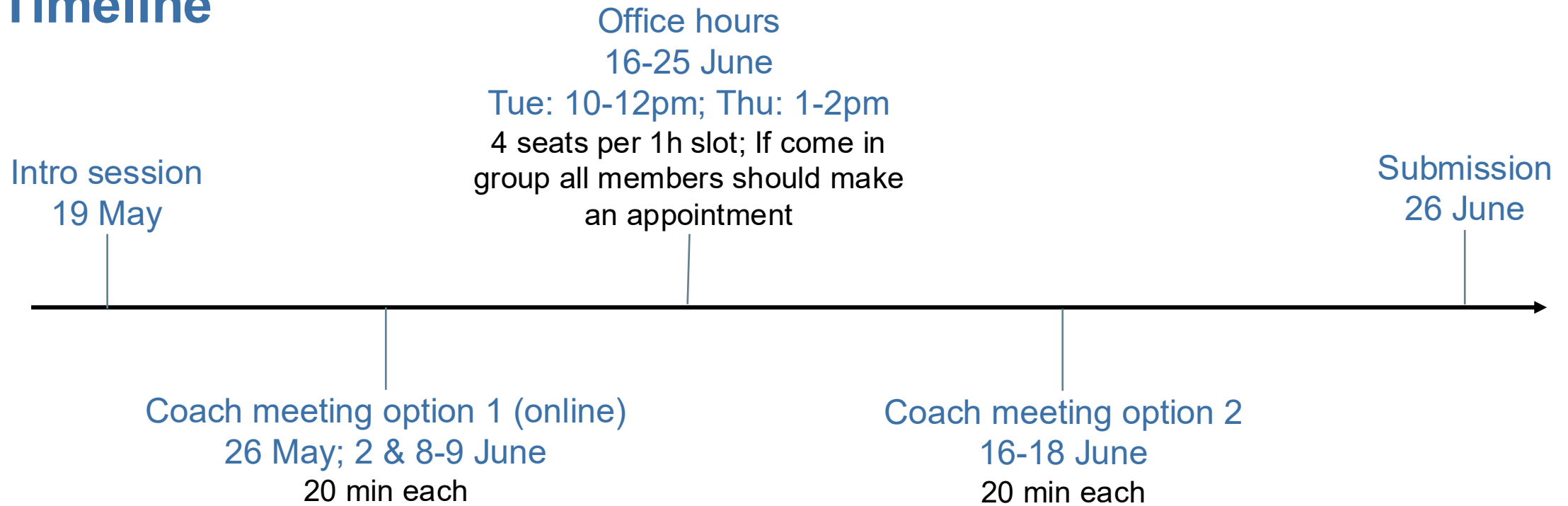
6

Q & A

Timeline



Timeline



Information:

- Book your appointments using [this link](#); one member each group (except for office hours)
- Can be online/in-person for option 2 & office hours
- Urgent issues/questions/slots unavailable: email to make an appointment
- Avoid emailing me texts that take me more than 10 min to think and respond

Assignment overview

- **Code that works and implements the key features of your business plan**
 - Well-structured .claude/.codex/.gemini/.agents and CLAUDE.md/CODEX.md/GEMINI.md/AGENTS.md
 - Code that implements your MVP; Documentation in form of a Markdown README file
 - Submission via a public GitHub repository link with git collaboration history
- **Video presentation**
 - Tech pitch; **Investor oriented**
 - Briefly describe the software architecture
 - Walk through the key code sections mapped to business plan's key features (and what they fail to address)
 - Demo: how the software is deployed and how users interact with it
 - End it with development setup: ~~programming languages~~ coding agent(s) used (e.g. Cursor, Claude Code, Copilot) and why, and the key ways you orchestrate your agents

Requirements

- **Mandatory**

- Submission via a public GitHub repository link (**Put the link in the comment field when you upload your video**)
 - **Refresh the page and confirm the video was submitted successfully**
- Readme documentation & GitHub repo: concise and simple; How to run it; What are requirements; licenses & copyright information, topic labels etc.

- **Optional**

- Other documentation (for data, processing workflow, algorithm, etc.)
- Describe how you generate synthetic data, if applicable
- Well-established agent orchestration
- A flow chart identifying which parts of your BP are implemented and which are not

Requirements

- **Feel free to...**
 - Can use any AI tools (or not)
 - Can use 3rd party services or libraries
 - Can use others' public code (careful about licenses)
- In real life: you do not need to reveal your AI setup

Grading

- Performance: the main features work and work well
- Presentation (highest weight): verbal communication, informativeness, structure, showcases
 - Avoid reading slides
- Code quality and documentation: concise and precise; I can run your program without extra effort
- AI instructions: relevant to the architecture (New)
- Collaboration and version control with Git (New)
- Other features and user experience (is it frustrated to use your product?)

Identifying key features

- **Identify and implement ALL key features**
 - The core value-enhancing process
 - The part where technology is the most essential
 - Examples: data; algorithm (input → output, performance metrics); UI/UX; intermediation between parties; combining frontend and backend
- **Example – P2P loan lending platform (what was missing)**
 - Shown: a list of loans with static data and a predicted default rate
 - Missing: the multi-party flow where borrower submits an application and lender views it from their side with information provided by the algorithms
 - End goal: the lender can actually use the tool to decide which loan to fund rather than just viewing numbers

Version control with Git

- Version control = track edits, revert mistakes, **collaborate safely**
 - Keep track of your changes (git add, git commit)
 - Backup and collaboration (git push/pull)
 - Rolling back when necessary (git revert)
- GitHub = cloud-based platform that hosts your code in repositories (project folders)
- Not suitable for binary files
- **Create an empty repository and push to remote GitHub repository before you start!!**
 - In the past student accidentally overwrote everything when trying to push/upload to GitHub
- Use AI to manage your repository
 - But do you want the “co-authored by <AI name>”?

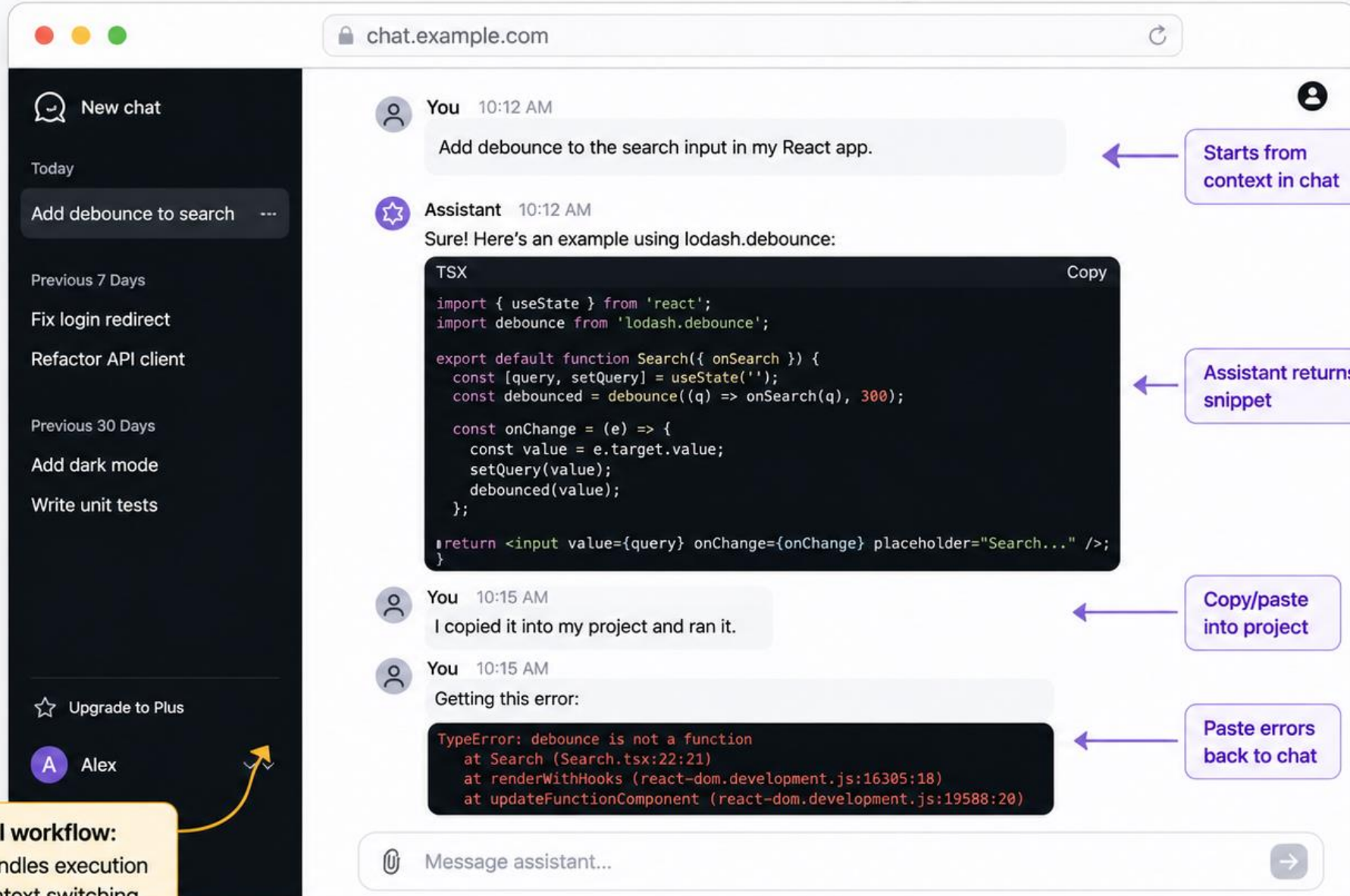
Implementation with AI

- **Why and how to use AI?**
 - Skill demand shift in the age of AI tools
 - Interview assessment changes focus from correctness to real understanding and independent thinking (e.g., Google recently)
 - Why did you accept certain suggestions?
 - Why did you reject others?
 - How did you decide when AI helped versus when it created more work?

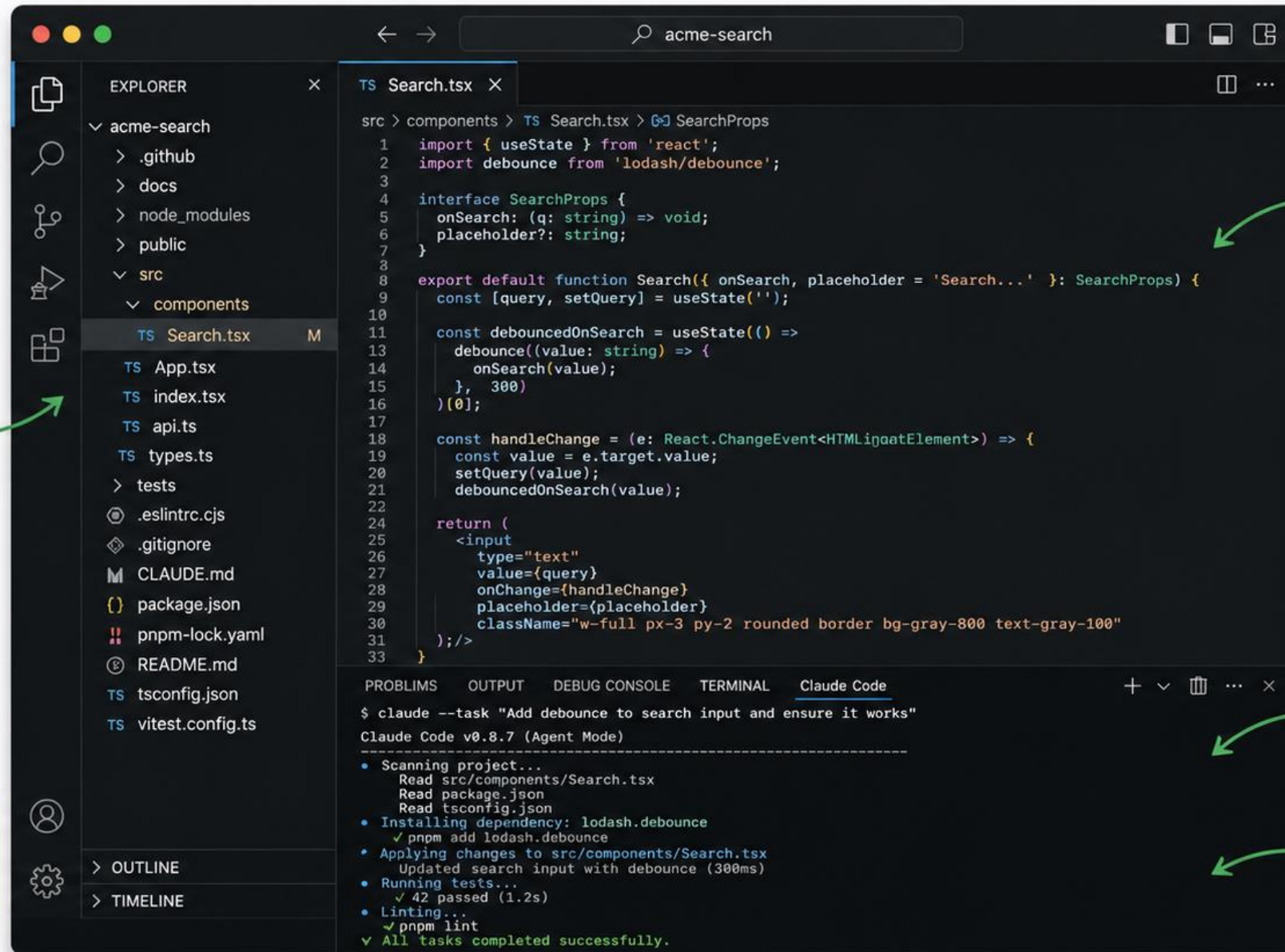
Implementation with AI

- **Why and how to use AI?**
 - Skill demand shift in the age of AI tools
 - Interview assessment changes focus from correctness to real understanding and independent thinking (e.g., Google recently)
 - Why did you accept certain suggestions?
 - Why did you reject others?
 - How did you decide when AI helped versus when it created more work?
- **Next questions: what AI, which AI provider, and how to implement**
 - Quick survey: Have you ever used AI Chatbot? AI agents?

Chat coding



Agentic coding



Lives inside
the folder

Reads and
edits files

Runs and fixes
errors automatically

Maintains project
structure and
organization



Autonomous workflow:
agent handles execution,
iteration, and project context

Chat coding vs. agentic coding

Aspect	Chat coding	Agentic coding (e.g., Claude Code)
Workflow	Chat, paste snippet, run, copy errors back.	Reads, edits, runs, fixes, iterates autonomously .
Project context	Starts each session from zero.	Lives in your folder; understands the full project .
Execution	You run code and manage files manually.	AI executes via tools (read/write/bash/grep) and reorganizes files.
Error handling	You paste errors back to the chat.	Detects errors at runtime, edits, and reruns automatically.
Setup	Plain chatbot session.	Folder-scoped session with persistent instructions (e.g., CLAUDE.md).

Claude code cheatsheet

Important concepts

~/ .claude

AI-instructional context — auto-loaded each session

CLAUDE.md

KEY

A .md file read at the start of every session

Skills / Rules

KEY

Reusable instructions for repeated prompts...Yet may be ignored

Plan / Auto mode

Toggle with Shift + Tab

Subagents

Delegate scoped tasks to focused agents

Reference: support.claude.com/en/articles/14553413-claude-code-cheatsheet

RSM
Erasmus

claude — important commands

Bootstrap your project

\$ **/init**

→ Explore the codebase & generate a starter **CLAUDE.md**

Tune model thinking

\$ **/effort**

→ How hard the model thinks

Manage context & cost

\$ **/compact /clear /usage**

→ Minimize cost & optimize context

Quick status

\$ **/statusline**

→ Model, context, usage at a glance

Which AI provider (as of early May 2026)?

Tool	Good at	Key features	Quota	Pricing	Latest model	Harness rating
 Claude Code	Agentic coding & reasoning-heavy fixes	Repo editing, tests, commits, subagents, hooks, MCP	Medium–very high	Pro \$20/mo; Max from \$100/mo	Opus 4.7	Claude Code + Opus 4.6: 58%
 Codex	Agentic coding & cloud tasks	CLI, IDE, web, cloud tasks, code review, subagents, MCP	Medium–very high	Plus \$20/mo; Pro from \$100/mo	GPT-5.5	Codex CLI + GPT-5.5: 82%
 Gemini CLI	Long-context work, automation, Google-grounded coding	Open-source CLI, 1M context, Search grounding, shell/file tools, MCP	Very generous free tier	Free; Pro \$19.99/mo; Ultra \$249.99/mo	Gemini 3.1 Pro	Gemini CLI + 3 Flash: 47.4%

Use AI tools responsibly

- Clear boundaries between what should be generated by AI and what should be generated by you (this also applies to myself)



COGNITIVE SURRENDER

Shaw & Nave · Wharton, U. of Pennsylvania

SETUP Volunteers answered hard questions with AI help. Researchers secretly seeded errors into some AI responses.

FINDING With correct AI, users beat the no-AI control. With wrong AI, they did far worse. They stopped checking.

TAKEAWAY AI's confident tone triggers “cognitive surrender.” Always verify before you trust.



ON-DEMAND ≠ BETTER LEARNING

Poulidis et al. · INSEAD

SETUP 200+ chess-club students trained on an AI platform. Some got hints at fixed moments, others on demand.

FINDING On-demand users gained less than half the skill of the fixed-hint group.

TAKEAWAY Unlimited access erodes learning. Build in AI-free practice to grow real skill.

Setup for agentic coding

- Editors: VS Code, Cursor, PyCharm, etc.
- VS Code extensions (+illustration on how to install and use):
 - Jupyter
 - Git Graph (for version control)
 - Claude Code/Codex

Q&A

- Any questions?

Thank You!